



**UNIVERSITY
OF MALAYA**



WIX1002 : FUNDAMENTALS OF PROGRAMMING

VIVA 3: CLASS

SEM 1 2025 / 2026

LECTURER NAME:

DR. NURUL BINTI JAPAR

GROUP MEMBERS:

1. JOSEPHINE DING JIE YU (25005876)
2. NG YUE QHI (25005961)
3. NG GEOK LIU (25005946)
4. NG SHAO ERN (25006089)
5. JASMINE CHIN JIA YEE (25006180)
6. JASMINE CHIN YING HUI (25005998)

Question 1: Define a Class: Role

Section 1: Problem

In the magical kingdom, there exists an ancient and powerful organization — the Arcane Guild. Every member of the guild has a name, age, race, and some magical power. These details should be protected using encapsulation, while still allowing the program to create members with default or specified values and perform a basic magical action.

Section 2: Solution

To solve this, a Java class called **Role** is created in the package **com.guide.roles.Role**. The class uses private variables to store the member's information. It includes a no-argument constructor to assign default values and a parameterized constructor to initialize all attributes. Getter and setter methods are implemented for each variable to provide controlled access and allow safe modification of the data when needed.

Lastly, a method called **performAction()** is added to represent the basic behavior of a guild member by printing a message that shows the role performing a magical action. This overall design ensures the class is well-structured, flexible, and follows object-oriented programming principles.

Section 3: Source code

```
package com.guide.roles.Role;

public class Role {
    private String name;
    private Integer age;
    private String race;
    private Double mana;

    public Role() {
        name=null;
        age=0;
        race=null;
        mana=0.0;
    }
}
```

```

public Role(String n,Integer a,String r,Double m){
    this.name=n;
    this.age=a;
    this.race=r;
    this.mana=m;
}

public String getName(){
    return name;
}

public Integer getAge(){
    return age;
}

public String getRace(){
    return race;
}

public Double getMana(){
    return mana;
}

}

public void setName(String n){
    this.name=n;
}

public void setAge(Integer a){
    this.age=a;
}

public void setRace(String r){
    this.race=r;
}

public void setMana(Double m){
    this.mana=m;
}

public void performAction(){
    System.out.println(name+" is performing a magical action.");
}

}

```

Question 2: Test Class: RoleTest

Section 1: Problem

The problem requires creating a RoleTest class which is a test class to verify the functionality of the Role class by creating objects, modifying their data and displaying their details. This ensures that the Role class functions correctly and that encapsulation, constructors, setters and getters work as expected.

Section 2: Solution

To solve this problem, I created a main method and instantiated the Role class twice. Object 1 is created using the no-arg constructor, then populated using setter methods. The setter methods are used to modify attributes of the first role. Object 2 is created using the parameterized constructor with data passed directly. Finally, I called the performAction() method and printed all attributes using getter methods to verify the data is correct.

Section 3: Sample Output

```
run:
Alvin is performing a magical action.
Tharok is performing a magical action.

--- Role Details ---
Name: Alvin
Age: 120
Race: Elf
Mana: 300.5
-----
Name: Tharok
Age: 45
Race: Orc
Mana: 50.0
-----
BUILD SUCCESSFUL (total time: 0 seconds)
```

Section 4: Source code

```
1  package com.guild.roles.RoleTest;
2  import com.guild.roles.Role.Role;
3
4  public class RoleTest {
5
6      public static void main(String[] args) {
7
8          // Requirement 1: Create Role objects using both constructors
9
10         // Object 1: Using No-Argument Constructor
11         Role role1 = new Role();
12
13         // Requirement 2: Use setter methods to modify attributes for Object 1
14         role1.setName("Alvin");
15         role1.setAge(120);
16         role1.setRace("Elf");
17         role1.setMana(300.5);
18
19         // Object 2: Using Parameterized Constructor
20         Role role2 = new Role("Tharok", 45, "Orc", 50.0);
21
22         // Requirement 3: Call performAction() for each object
23         role1.performAction();
24         role2.performAction();
25
26         System.out.println("\n--- Role Details ---");
27
28         // Requirement 4: Print out all role details
29         printDetails(role1);
30         printDetails(role2);
31     }
32
33     // Helper method to print details cleanly
34     public static void printDetails(Role r) {
35         System.out.println("Name: " + r.getName());
36         System.out.println("Age: " + r.getAge());
37         System.out.println("Race: " + r.getRace());
38
39         System.out.println("Mana: " + r.getMana());
40         System.out.println("-----");
41     }
```

Question 3: Class Definition Requirements

Section 1: Problem

The task is to design and implement a Java class called MagicShield that models a magical shield used in battle. The shield has physical attributes (radius and thickness) and a magical attribute (element type). These attributes determine its defensive power and mana cost. The class must use encapsulation for its instance and static variables and provide constructors. It will implement instance methods to calculate defense power and mana cost while using static methods to calculate total shields created, check whether size of the shield is valid, calculate defense power and mana cost. The magic shield info including element type, radius, thickness, defense power and mana cost would be printed as output.

Section 2: Solution

To solve this, we have to declare encapsulated instance variables (radius, thickness and element type) and encapsulated static variables (DEFAULT_RADIUS, DEFAULT_THICKNESS and shield count). Static variables are to declare the default value for the variables while shieldCount variable is to calculate the number of shields (object) created. We will create two types of constructors which are no-argument constructor that use default values for each variable and parameterized constructor that accept user input (parameters) to change the value of each variable. Next, we will add accessors and mutators for each field. Accessor/getter allows the user to access the encapsulated values while the mutator/setter accepts assigned values from the object and check whether they follow the rules for the values of each variable or the system will throw an IllegalArgumentException. Next, we compute instance methods such as calculateDefensePower() and calculate manaCost() to calculate defense power and mana cost for each shield. We add static methods like getShieldCount() to return the total number of shields created, isValidSize() to validate shield size as well as calculateDefensePower() and calculateManaCost() to compute defense power and mana cost directly from any input. Then, will create an object in the Main method and print it by calling toString() method in order to display the magic shield info.

Section 3: Sample Input & Output

```
[MagicShield Info]
Element Type: Fire
Radius: 2.5 m
Thickness: 7.0 cm
Defense Power: 19.25
Mana Cost: 39.0
```

Section 4: Source Code

```
package com.maplestory;
public class Main{
    public static void main(String args[]) {
        //create object using parameterized constructor
        MagicShield s1=new MagicShield(2.5,7.0,"Fire");
        //Call instance methods
        System.out.println(s1.toString());}

    class MagicShield{
        //instance variables
        private double radius; //shield radius in meters
        private double thickness; //shield thickness in centimeters
        private String elementType; //elemental attribute of the shield

        //static variables
        private static final double DEFAULT_RADIUS = 1.0;
        private static final double DEFAULT_THICKNESS = 5.0;
        private static int shieldCount = 0;

        //no-argument constructor
        public MagicShield(){
            radius=DEFAULT_RADIUS;
            thickness=DEFAULT_THICKNESS;
            elementType="Neutral";
            shieldCount++;
        }
        //parameterized constructor
        public MagicShield(double r, double t, String e){
            setRadius(r); //to make sure it follows the rule inside setter method and not returning illegal value
            setThickness(t);
            setElementType(e);
            shieldCount++;
        }
    }
}
```

```

//instance methods
public double calculateDefensePower(){
//elementCoefficient table using switch case
    double elementCoefficient;
    elementCoefficient = switch (elementType) {
        case "Fire" -> 1.1;
        case "Ice" -> 1.2;
        case "Light" -> 1.3;
        case "Dark" -> 1.4;
        default -> 1.0;
    };
    double defensePower=(radius*thickness)*elementCoefficient;
    return defensePower;
}

public double calculateManaCost(){
double manaCost = radius*10 + thickness*2;
return manaCost;
}

```

```

//static method
//total shields created
static int getShieldCount() {
    return shieldCount;
}

//check if radius/thickness is valid
static boolean isValidSize(double size){
return (size>=0);
}

//calculate defense power directly from input
static double calculateDefensePower(double radius, double thickness, String elementType){
    double elementCoefficient;
    elementCoefficient = switch (elementType) {
        case "Fire" -> 1.1;
        case "Ice" -> 1.2;
        case "Light" -> 1.3;
        case "Dark" -> 1.4;
        default -> 1.0;
    };
    double defensePower=(radius*thickness)*elementCoefficient;
    return defensePower;
}

//calculate mana cost directly from input
static double calculateManaCost(double radius, double thickness){
double manaCost = radius*10 + thickness*2;
return manaCost;
}

```

```

//encapsulation method
//accessors
public double getRadius(){
    return radius;
}
public double getThickness(){
    return thickness;
}
public String getElementType(){
    return elementType;
}
//mutators
public void setRadius(double radius){
    if(!isValidSize(radius))
        throw new IllegalArgumentException("Invalid radius");
    this.radius=radius;
}
public void setThickness(double thickness){
    if(!isValidSize(thickness))
        throw new IllegalArgumentException("Invalid thickness");
    this.thickness=thickness;
}
public void setElementType(String type){
    if (type==null||type.equals(""))
        elementType="Neutral";
    else
        elementType=type;
}

//toString method
public String toString(){
    return String.format(
        ""
        [MagicShield Info]
        Element Type: %s
        Radius: %.1f m
        Thickness: %.1f cm
        Defense Power: %.2f
        Mana Cost: %.1f
        "" ,elementType,radius,thickness,calculateDefensePower(),calculateManaCost()
    );
}
}

```

Question 4: Test Class: MagicShieldTest

Section 1: Problem

We are testing the MagicShield class in a MapleStory-style game. Each shield has:

- radius (m), thickness (cm) and elementType (Fire, Ice, Light, Dark, Neutral).
- Shields can calculate defense power and mana cost.
- Validation must prevent negative or invalid inputs.
- Static methods allow calculation without creating objects.
- A static counter tracks the total number of shields created.

Requirements for testing:

1. Create multiple shields using both no-argument and parameterized constructors.
2. Test setters with valid and invalid values.
3. Calculate and print defense power and mana cost.
4. Use static methods to calculate properties without creating objects.
5. Print total number of shields created.
6. Test edge cases: null, 0, negative values.

Section 2: Solution

Step 1: Creating Objects

- shield1 uses the no-argument constructor, initializing default radius, thickness and elementType.
- shield2 and shield3 use the parameterized constructor with custom values.

Step 2: Testing Setters

- Setters are tested with valid values to update shield attributes.
- Setters are also tested with invalid values (negative radius or thickness) to ensure IllegalArgumentException is thrown.

Step 3: Calculating Shield Properties

- Each shield's defense power and mana cost are calculated using instance methods.
- toString() provides a formatted representation including element type, radius, thickness, defense power and mana cost.

Step 4: Using Static Methods

- calculateDefensePower() and calculateManaCost() can compute shield properties without creating an object.
- This allows flexible calculations for arbitrary radius, thickness and elementType.

Step 5: Verifying Total Shields Created

- getShieldCount() confirms how many shields were instantiated during the test.

Step 6: Edge Case Handling

- Shields with zero or null values are handled gracefully.
- Negative values trigger exceptions, which are caught and logged.

Section 3: Sample Input & Output

Input: Refer to source code in Section 4

Output:

```
Exception caught: Invalid radius
Exception caught: Invalid thickness

Shield 1:
[MagicShield Info]
Element Type: Neutral
Radius: 1.0 m
Thickness: 5.0 cm
Defense Power: 5.00
Mana Cost: 20.0

Shield 2:
[MagicShield Info]
Element Type: Light
Radius: 3.5 m
Thickness: 0.0 cm
Defense Power: 0.00
Mana Cost: 35.0

Shield 3:
[MagicShield Info]
Element Type: Ice
Radius: 3.0 m
Thickness: 6.0 cm
Defense Power: 21.60
Mana Cost: 42.0

Using static methods:
Defense Power: 28.0
Mana Cost: 50.0

Total Shields Created: 3

Shield 4:
[MagicShield Info]
Element Type: Neutral
Radius: 0.0 m
Thickness: 0.0 cm
Defense Power: 0.00
Mana Cost: 0.0
Exception in thread "main" java.lang.IllegalArgumentException: Invalid thickness
    at com.maplestory.MagicShield.setThickness(MagicShield.java:43)
    at com.maplestory.MagicShield.<init>(MagicShield.java:21)
    at com.maplestory.MagicShieldTest.main(MagicShieldTest.java:59)
/Users/shaoern/Library/Caches/NetBeans/12.5/executor-snippets/run.xml:111: The following error occurred while executing this line:
/Users/shaoern/Library/Caches/NetBeans/12.5/executor-snippets/run.xml:68: Java returned: 1
BUILD FAILED (total time: 0 seconds)
```

Section 4: Source Code

```
package com.maplestory;

public class MagicShieldTest {
    public static void main(String[] args) {

        // Create objects using different constructors
        MagicShield shield1 = new MagicShield();
        MagicShield shield2 = new MagicShield(2.5, 7.0, "Fire");
        MagicShield shield3 = new MagicShield(3, 6, "Ice");

        // Test setters with valid values
        try {
            shield2.setRadius(3.5);
            shield2.setThickness(0);
            shield2.setElementType("Light");
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        }

        // Test setters with invalid values
        try {
            shield3.setRadius(-1.0);
        } catch (IllegalArgumentException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }

        try {
            shield3.setThickness(-8);
        } catch (IllegalArgumentException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }

        // Calculate and print defense power and mana cost
        System.out.println("\nShield 1: ");
        System.out.println(shield1);
        System.out.println("\nShield 2: ");
        System.out.println(shield2);
        System.out.println("\nShield 3: ");
        System.out.println(shield3);

        // Use static methods to calculate shield properties for arbitrary parameter combinations
        double defensePower = MagicShield.calculateDefensePower(4, 5, "Dark");
        double manaCost = MagicShield.calculateManaCost(4, 5);

        System.out.println("\nUsing static methods: ");
        System.out.println("Defense Power: " + defensePower);
        System.out.println("Mana Cost: " + manaCost);

        // Output total number of shields created
        System.out.println("\nTotal Shields Created: " + MagicShield.getShieldCount());

        // Test edge cases (null, 0, negative values)
        MagicShield shield4 = new MagicShield(0, 0, null);
        System.out.println("\nShield 4: ");
        System.out.println(shield4);

        MagicShield shield5 = new MagicShield(0, -8, null);
        System.out.println("\nShield 5: ");
        System.out.println(shield5);
    }
}
```

Question 5

Section 1: Problem

There is an ancient Arcane Academy that trains future mages and alchemists. Each student has a name, age and a mana level. The academy needs to know the total number of students and determine whether two students are exactly the same based on their name and age.

Section 2: Solution

A base class name Person is created. In this base class variables are all private and divided into instance variables and static variables. Variables name and age are classified as instance variables while variable DEFAULT_AGE and personCount are classified as static variables. DEFAULT_AGE is set to 18 while personCount is set to 0.

Then, a non-argument constructor and a parameterized constructor is created. In the non-argument constructor name is initialized as “Unknown Student” while age is initialized as DEFAULT_AGE. In the parameterized constructor, it accepts name and age and assigns to the instance variables. Both of the constructors will increase personCount when called.

Encapsulation methods are also created for all instance variables. getName() and getAge() return their values while setName(String name) and setAge (int age) modify the values of the variables. A Illegal Argument Exception error will be thrown if the age received is negative.

There are also two class methods which are getPersonCount() and compareTo(Person other). getPersonCount() returns the value of variable personCount. compareTo(Person other) returns a boolean value after checking age and name of other and the object calling this method.

Section 3: Source code

```
package com.magical.people.Person;
public class Person{
    private String name;
    private int age;

    private static final int DEFAULT_AGE = 18;
    private static int personCount = 0;

    public Person(){
        name = "Unknown Student";
        age = DEFAULT_AGE;
        personCount ++;
    }

    public Person(String name, int age){
        this.name = name;
        this.age = age;
        personCount ++;
    }

    public String getName(){
        return name;
    }

    public int getAge(){
        return age;
    }

    public void setName(String name){
        this.name = name;
    }
}
```

```
] public void setAge(int age) {  
]     if (age >= 0) {  
]         this.age = age;  
-     } else {  
-         throw new IllegalArgumentException("Age cannot be negative");  
-     }  
- }  
  
] public static int getPersonCount() {  
-     return personCount;  
- }  
  
] public boolean compareTo(Person other) {  
-     return name.equals(other.name) && age == other.age;  
- }  
}
```

Question 6 : Test Class: PersonTest

Section 1: Problem

This question requires the development of a test program to verify the correctness of the Person class. The program must test object creation, comparison between two Person objects, and validation of age input. It should also confirm that the static variable used to count the total number of Person objects is functioning correctly. The test program is expected to handle invalid input properly by using exception handling and to display appropriate outputs that demonstrate the behavior of the class methods..

Section 2: Solution

To solve this problem, a test class named PersonTest is implemented. In the program, several Person objects are created using the available constructors, and their details are displayed to verify correct initialization. The compareTo() method is then used to compare different Person objects to determine whether they represent the same individual. The setAge() method is tested by attempting to assign a negative age value inside a try-catch block, ensuring that an exception is thrown and handled correctly. Finally, the static method that returns the total number of Person objects is called and displayed, confirming that the object counter is updated properly. This test program effectively demonstrates the correctness and reliability of the Person class.

Section 3: Input & Output

```
Age of person 1: 20
Age of person 2: 28
Age of person 3: 20
Age of person 4: 18

After update age of person 4
Age of person 4: 20

Caught exception: Age cannot be a negative value

Comparison:
Person1 vs Person2: false
Person1 vs Person3: true
Person1 vs Person4: false

Total students: 4
-----
BUILD SUCCESS
-----
```

Section 4: Source Code

```
9 package com.magical.people.PersonTest;
10 class PersonTest {
11     public static void main(String[] args) {
12         //Create person object using parameterized constructor
13         Person p1 = new Person("Ali", 20);
14         Person p2 = new Person("Lim", 28);
15         Person p3 = new Person("Ali", 20);
16         //Create person object using no-argument constructor
17         Person p4 = new Person();
18
19         System.out.println("Age of person 1: "+p1.getAge());
20         System.out.println("Age of person 2: "+p2.getAge());
21         System.out.println("Age of person 3: "+p3.getAge());
22         System.out.println("Age of person 4: "+p4.getAge());
23
24         //Call setAge() with valid age
25         p4.setAge(20);
26         System.out.println("\nAfter update age of person 4");
27         System.out.println("Age of person 4: "+p4.getAge());
28
29         //Call setAge() with invalid age
30         try{
31             p4.setAge(-2);
32         } catch(IllegalArgumentException e){
33             System.out.println("\nCaught exception: "+ e.getMessage());
34         }
35
36         //Call compareTo() to compare different students
37         System.out.println("\nComparison: ");
38         System.out.println("Person1 vs Person2: "+ p1.compareTo(p2));
39         System.out.println("Person1 vs Person3: "+ p1.compareTo(p3));
40         System.out.println("Person1 vs Person4: "+ p1.compareTo(p4));
41
42         //Call getPersonCount() to verify the total student count
43         System.out.println("\nTotal students: "+Person.getPersonCount());
44     }
45 }
```